Lecture 9 · Part II · Lecture notes

Training deep networks

Dataset: CIFAR-10

How to use these notes

The primary text for this lecture is Géron, Chapter 11. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 A deep stack that will not train **2**

1.1 The specification, and what it does not say . . . 2

2 Instrumenting it **2**

2.1 Probe 1, and why it was wrong . . . 3

2.2 Probe 2 — the backward pass . . . 3

3 Derivation: variance propagation through layers **4**

3.1 Two requirements, and they conflict . . . 4

3.2 Putting the activation back . . . 5

4 Six repairs, measured one at a time **5**

4.1 Initialisation, and why it is not enough . . . 5

4.2 The activation, and normalisation . . . 5

4.3 Clipping, optimisers and schedules . . . 7

5 The ablation **7**

5.1 The silent-failure catalogue so far . . . 8

6 The notebook **8**

7 Design exercise: a voice-assistant company's wake-word detector that stopped training **8**

8 Exercises **10**

Summary **11**

---

1 A deep stack that will not train

You can build a network and you own its training loop. So build a deep one — twenty hidden layers on CIFAR-10 — and train it exactly as Lecture 8 taught.

It does not train. The loss barely moves. Nothing raises, nothing is `NaN`, the harness is correct and the data is fine, and a shallower version of the same network trains without complaint.

Today we measure what goes wrong, derive why, and repair it — in that order, because the repair is unreadable until you have seen the measurement.

1.1 The specification, and what it does not say

3,072 inputs, 20 hidden layers of 100 units, a logistic activation on each, a linear head of 10 outputs. Cross-entropy, Adam at 0.001, batches of 128, 20 epochs.

Read that again and note what it does not mention: *how the weights start out*. That is not something being hidden — it is the ordinary case. Almost no specification mentions it.

A model that has learned nothing scores $\ln 10 = 2.3026$. Epoch 1 gives 2.3091 and epoch 20 gives 2.3036: the loss moved by 0.0055 in twenty epochs, and the test accuracy is **10.0%** — exactly what a single `return 3` would have bought.

Adding layers made it worse and past a point made it useless — which is not what capacity is supposed to do.

The loop is correct — it memorises 200 images on demand; the data is correct — a two-layer network on the same data reaches 39.6%; the metric is correct — counted over the set, in evaluation mode. The depth is the variable, and we have no account of why. Which is where Lecture 8's real purchase pays.

2 Instrumenting it

Backpropagation computes 500,210 partial derivatives on every batch. Under `MLPClassifier.fit()` every one was discarded; they are all still here in `p.grad`. Two probes, about ten lines each — without them the diagnosis is "it does not train" and there is no next step.

2.1 Probe 1, and why it was wrong

Probe 1 runs the modules by hand and records, after each logistic, the mean, the standard deviation over the whole tensor and a saturated fraction.

"Saturated" means the logistic's own derivative has collapsed — here, an output further than 0.45 from a half. Mean near a half, spread near 0.071, flat for twenty layers, nothing saturated. The forward signal looks fine.

Failure condition — the probe measured the wrong thing

`h.std()` spans the *whole tensor*, so it is dominated by the spread of the layer's random biases across units — which does not change with depth. It would read 0.07 in a network that had stopped computing entirely.
What carries information is the spread *down the batch*: how much a unit moves when the input does.

Hidden layer	1	5	10	20
`h.std()` — what we plotted	0.131	0.077	0.073	0.071
`h.std(dim=0).mean()` — the signal	1.3e−1	5.9e−5	3.0e−9	2.4e−17

Sixteen orders of magnitude. The forward signal *does* die, by a factor of 0.148 per layer — the order of the factor the derivation below predicts from the fan-in, and the rate the gradient vanishes at — and by layer 20 the network is no longer a function of its input.
The saturation column was no better. It asks whether $|h - \frac{1}{2}| > 0.45$ in a band whose standard deviation is 0.071 — a threshold 6.3 standard deviations away, and 3.4 even at layer 1. Nothing could ever have exceeded it. "0.000 at every depth" was arithmetic, not evidence.

2.2 Probe 2 — the backward pass

Eight batches rather than one, because a single batch of 128 is a noisy estimate of anything and a course that says so should not then quote one.

Layer 20 receives 1.45×10^{15} times what layer 1 does.

The shape of that line is the finding

It is *straight* on a logarithmic axis, and a straight line on a log axis is a geometric sequence. The attenuation is not one bad layer or a fault at the input: it is the same factor applied at every layer,

$$\frac{\|g_\ell\|}{\|g_{\ell+1}\|} \approx \rho \qquad \Longrightarrow \qquad \frac{\|g_1\|}{\|g_{20}\|} \approx \rho^{19}.$$

One number characterises the whole network, and here it is

$$\rho = 0.1593, \qquad \rho^{19} = 6.91 \times 10^{-16}.$$

Does training rescue it? We logged the per-layer norms every epoch, so this is a lookup rather than a guess: layer 1's gradient was 6.660e−17 at epoch 1 and 4.610e−17 at epoch 20. Twenty epochs of Adam did not move it into a range a learning rate of 0.001 could use.

Failure condition — an optimiser asked to supply missing information

Adam will compensate. It rescales, so the direction is what matters — and at 10^{-17} the direction is dominated by batch-to-batch noise. Worse, Adam's ε, default 10^{-8}, is *larger* than the gradients at the bottom of the stack. An optimiser can compensate for a badly scaled gradient; it cannot manufacture information the backward pass did not deliver.

What we cannot yet say is why that factor is 0.159 and not 1. That is a question about the weight matrices themselves, and it has an exact answer.

3 Derivation: variance propagation through layers

Drop the activation and look at the linear part: $z_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} a_j$. Three assumptions, stated so they can be checked: the weights are drawn independently with mean zero; they are independent of the inputs (true at initialisation, approximately true afterwards); and the inputs have mean zero and are uncorrelated.

All three are idealisations, which is fine — the point of an idealisation is that you can compute with it and then check the answer.

The one recursion

Each term $w_{ij} a_j$ has mean zero and the terms are uncorrelated, so the variances add:

$$\operatorname{Var}(z) = n_{\text{in}} \cdot \operatorname{Var}(w) \cdot \operatorname{Var}(a).$$

Everything in this lecture is a consequence of that line. Note its shape: the input variance is *multiplied* by a factor, which is what makes depth dangerous.

3.1 Two requirements, and they conflict

The forward pass wants the signal to arrive at layer 20 with the spread it had at layer 1:

$$\operatorname{Var}(z) = \operatorname{Var}(a) \iff n_{\text{in}} \operatorname{Var}(w) = 1 \iff \operatorname{Var}(w) = \frac{1}{n_{\text{in}}}.$$

The backward pass runs the same matrix *transposed*, $\bar{a} = W^{\mathsf{T}} \bar{z}$, so identical algebra with one index swapped:

$$\operatorname{Var}(\bar{a}) = n_{\text{out}} \cdot \operatorname{Var}(w) \cdot \operatorname{Var}(\bar{z}) \qquad \Longrightarrow \qquad \operatorname{Var}(w) = \frac{1}{n_{\text{out}}}.$$

The two agree only when $n_{\text{in}} = n_{\text{out}}$ — never on our first layer, 3,072 into 100. **Glorot** does not choose, but takes the harmonic mean:

$$\operatorname{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}},$$

a small error in both directions rather than a large error in one. Harmonic because the quantities being averaged are reciprocals of the fans, and on a square layer it is not a compromise at all — it reduces to $1/n$. On our hidden layers that is $\operatorname{Var}(w) = 0.0100$, a standard deviation of 0.1000, against the default's 0.0577. In code it is `nn.init.xavier_normal_`.

3.2 Putting the activation back

Between two linear maps sits φ, and the backward pass multiplies by its derivative pointwise, so

$$\rho = \sqrt{n_{\text{out}} \cdot \text{Var}(w) \cdot \mathbb{E}\left[\varphi'(z)^2\right]}\,.$$

Two factors we control — the shape of the matrix and the variance we initialise with — and one belonging to the activation.

The logistic's ceiling

$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \le \frac{1}{4}$ for *every* z, with equality only at $z = 0$. Not "usually small": bounded above by a quarter everywhere.
So the logistic multiplies ρ by at most 0.25 before any weight has been chosen. Even a perfectly initialised logistic stack cannot do better than $(1/4)^{19}$.

ReLU discards exactly half the variance instead: for symmetric zero-mean z, $\mathbb{E}[\text{ReLU}'(z)^2] = \frac{1}{2}$ — half the time the derivative is 1, half the time 0. A clean factor of two, and a factor of two can be paid for. **He** pays it by doubling the weight variance, $\text{Var}(w) = 2/n_{\text{in}}$, giving $\rho = \sqrt{n_{\text{out}}/n_{\text{in}}}$ — exactly 1 when the fans are equal. On our hidden layers that is $\text{Var}(w) = 0.0200$ and a standard deviation of 0.1414, against Glorot's 0.1000 and the default's 0.0577. Because the same factor ρ is applied at every layer, twenty layers multiply the gradient by ρ^{19} — which is how a per-layer factor of 0.1593 became one part in 1.45×10^{15}. Only $\rho = 1$ survives depth.

4 Six repairs, measured one at a time

4.1 Initialisation, and why it is not enough

Two lines per `Linear`: `nn.init.xavier_normal_(m.weight)` and `nn.init.zeros_(m.bias)`, or for ReLU `nn.init.kaiming_normal_(m.weight, nonlinearity="relu")`. Kaiming is He; the `nonlinearity` argument is what selects the factor of two, and its default is `"leaky_relu"` with slope 0.01 — near enough, but say what you mean.

	ρ	End to end	Test accuracy
Default, logistic	0.1593	1.4e+15	10.0%
Glorot, logistic	0.2531	2.2e+11	10.0%

Nearly four orders of magnitude better, and still eleven orders of attenuation — which is hopeless. The measured ρ is now 0.253, which is 0.25: the logistic's own ceiling, with the weight scale perfectly neutralised. *You cannot initialise your way out of the activation function.*

4.2 The activation, and normalisation

Every requirement on φ comes from the derivation: φ' should reach 1 somewhere and not be bounded below it; φ should not saturate on both sides; and the output should be roughly centred so assumption 3 holds at the next layer. ReLU satisfies the first two and fails the third, and is still

the right default — though it has its own failure mode, a unit whose pre-activation is negative for every input has gradient exactly zero forever. Leaky ReLU, ELU and SELU all repair that the same way, by giving the negative side a non-zero slope.

Every table in this section is one run on one seed. The ablation in the next section repeats each configuration over five seeds and reports the mean, which is why the same ReLU + He network reads 41.80% below and 41.00% there.

Activation	Initialisation	Test accuracy
logistic	Glorot	40.39%
tanh	Glorot	40.57%
ReLU	He	36.59%
Leaky ReLU	He	37.71%
ELU	He	43.98%
SELU	Glorot	42.70%

Spread 7.39 points — and the logistic is *not last*. All six carry batch normalisation, which is why it survives: normalisation puts the pre-activations back where its derivative is largest, at every step. Without normalisation, moving off the logistic to ReLU with He took one run from 10.00% to 41.80%, 31.80 points. With it, the choice of activation is worth 7.39.

Batch normalisation standardises each layer's inputs and then learns what scale they should have had,

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma \hat{z} + \beta,$$

where μ_B and σ_B are computed over the batch and γ, β are learned. The second step is what stops it being a straitjacket — the network can undo the normalisation if it needs to. Initialisation gets ρ right once; normalisation makes it right at every step.

On the ReLU + He network	Test accuracy	Parameters
No normalisation	41.80%	500,210
Batch normalisation	36.59%	504,210
Layer normalisation	41.67%	504,210

Batch normalisation made it *worse*, by 5.21 points on this run (by 4.33 over five seeds, in the ablation below). The parameter count is the claim that travels: either normalisation adds the same few thousand parameters to half a million, and one more pass over every layer's activations on every step.

Failure condition — the bill batch normalisation carries

Its statistics depend on the other images in the batch, so at training time the prediction for one image depends on which images happened to be batched with it, while at evaluation it uses running averages. Those are two different functions.

That is Lecture 8's `model.eval()` failure again — and now much harder to catch. Dropout wobbles between calls, so running the evaluation twice exposes it; batch normalisation in training mode is deterministic for a fixed batch, so the same trick finds nothing. If there is a normalisation layer, ask whether `eval()` was called: running it twice will not tell you.

4.3 Clipping, optimisers and schedules

Gradient clipping rescales the whole gradient vector when its norm exceeds a threshold, preserving the direction and bounding the length — `clip_grad_norm_(net.parameters(), 1.0)` between `backward()` and `step()`. It is a defence against $\rho > 1$, and we have $\rho < 1$. Look at the distribution of norms before choosing a threshold: a clip below the median silently turns your optimiser into something else. Applied alone to the failing network it buys nothing at all. Added to the repaired network in the five-seed ablation below, it took the mean from 36.67% to 41.01%, +4.34 points, but the median step norm with He is 3.38 and the threshold was 1.0 — so it acted as a step-size limiter rather than a safety net. *A useful effect obtained for the wrong reason*; if you want a smaller step, set a smaller learning rate and say so.

Comparing optimisers on the broken network would have measured nothing. On the repaired one, one run each: SGD 27.6%, momentum 33.5%, Nesterov 33.9%, RMSProp 36.2%, Adam 36.6%, AdamW 37.8% — the first three at a learning rate of 0.01, the adaptive three at 0.001, since the same rate would not be a fair comparison. The spread between best and worst is far smaller than the effect of the initialisation — order your effort by the measurement, not by the novelty.

One step size cannot suit both the start of training and the end. Of the book's five schedules three are worth having: *exponential*, multiplying by a constant each epoch; *cosine*, annealing smoothly to zero over the run; and *1-cycle*, rising to a peak and then falling well below the starting value. All three need to know how long the run is, which is why they take `total_steps` and why changing the epoch count silently changes the schedule. Constant 40.31%, cosine 39.84%, 1-cycle 42.81% on a twenty-epoch run — on one seed each, and in the ablation the same schedule is worth +1.05 points against a five-seed spread of ±1.20, so read that as a shape rather than a ranking.

5 The ablation

Each row adds one change to the row above. Five seeds a row.

Configuration	Test accuracy	sd	Change
the failing network, unchanged	10.00%	±0.00	—
+ Glorot initialisation	10.00%	±0.00	+0.00 (noise)
+ ReLU and He initialisation	41.00%	±0.53	**+31.00**
+ batch normalisation	36.67%	±0.65	−4.33
+ gradient clipping	41.01%	±0.74	+4.34
+ a 1-cycle schedule	42.06%	±1.20	+1.05 (noise)
+ dropout 0.1	33.10%	±1.00	−8.96

The third column is the deliverable, and the fourth is what makes it readable. Typical spread, the mean of the sd column, is 0.59 points; one step is worth +31.00, more than fifty times that. The 1-cycle schedule does not clear its own spread — +1.05 against ±1.20. The notebook names the 1-cycle row as the best, at 42.06%, 1.05 points ahead of the runner-up, gradient clipping: the best row, but not a winner to build on. The last row is dropout, the layer from Lecture 8 that zeroes a random fraction of units in training: a regulariser, added to a network that is not overfitting.

The largest single step, ReLU with He, is worth +31.00 points; the best row against the start, +32.06; the worst single step, dropout, −8.96; the last row against the start, +23.10.

The answer to "should I add batch normalisation?"

One change is worth more than the whole ladder. The question is malformed until you know which failure you have — ours was $\rho \neq 1$, and only the repairs that address ρ paid.

5.1 The silent-failure catalogue so far

Failure	Diagnosed in	Measured cost
Missing `optimizer.zero_grad()`	Lecture 8	61 points
Metric averaged per batch	Lecture 8	0.067 points
A deep stack on default initialisation	today	31.00 points

Every one ran to completion and printed a plausible number. *The last is the first in which no line of code is wrong.*

6 The notebook

What to change

Set the initialisation back to the default and re-run the gradient probe. The straight line on a log axis comes back, and its slope is the per-layer factor the derivation predicts — so you will have closed the loop from measurement to theory to measurement yourself.

7 Design exercise: a voice-assistant company's wake-word detector that stopped training

The lecture closes with a design exercise in the manner of an ML system-design interview, on the system behind a company's product and on the method this lecture taught: probe, derive, repair, prove.

The task. A voice-assistant company ships a wake-word detector on every speaker and earbud it sells: a small network that reads each short window of audio and decides whether the user said the wake word. A miss makes the user repeat themselves; a false wake sends a stranger's kitchen to the cloud, which is a privacy complaint and a regulator's letter. The team redesigned it deeper for accuracy, and the new stack does not train: the loss sits at the value of a model that has learned nothing, nothing raises, nothing is `NaN`. Given: labelled audio windows from many speakers, the deeper model, a training loop the team owns, and the shallow detector that ships today and trains with the same loop. A stack of tens of layers, small enough to run on a battery, and runs on a subset cheap enough that many can be made. The output serves the engineer with the failing run, the reviewer who has to believe the repair, and the update that goes over the air to millions of devices.

1. **Frame it honestly.** The deliverable is a diagnosis — which failure — and one repair with a row of evidence, not a list of things tried. Establish the controls before touching the model:

the loop is correct, since the shipped shallow detector trains with it; the data is correct; the metric — misses and false wakes, counted over the set in evaluation mode — is the one the product tracks. Fix one seed and one subset for every run, with the shallow detector as the control. *Pitfall:* "does not train" is treated as a hyperparameter problem and a learning-rate grid is started on the cluster before anything is measured. The failing network reads the same at every learning rate, because the gradient reaching the bottom of the stack has been attenuated to noise, and no scalar multiplies noise into information.

2. **Probe the forward pass, and make sure the probe can fail.** Run the modules by hand on a few hundred audio windows and record, after each activation, the spread down the batch — how much a unit moves when the window does — layer by layer, on a log axis. *Pitfall:* the spread over the whole tensor is dominated by the random biases across units, which do not change with depth, so it reads the same in a network that has stopped listening; and a saturation count whose threshold sits many standard deviations outside the band returns zero at every depth, arithmetic rather than evidence. Run every probe on the failing stack and on the shipped detector before trusting it.

3. **Probe the backward pass.** After `backward()`, the norm of `weight.grad` for every linear layer, averaged over several batches, on a log axis against the layer index. A straight line is a geometric sequence: the same factor ρ at every layer, compounding over the depth, and only ρ near one survives it. Log it every epoch, so "does training rescue it?" is a lookup. *Pitfall:* one batch is a noisy estimate of anything; and "Adam will compensate" — it rescales, so the direction is what matters, and at the bottom of the stack the direction is batch-to-batch noise, with Adam's ε larger than the gradient. An optimiser cannot manufacture information the backward pass never delivered.

4. **Initialisation matched to the activation.** One layer multiplies the variance by $n\,\mathrm{Var}(w)$ — fan-in forwards, fan-out backwards; Glorot takes the harmonic mean of the two demands, and He doubles it for ReLU, which discards half the variance. With the activation put back, ρ is the square root of the fan-out times the weight variance times the mean squared derivative of the activation. Re-run the backward probe after the change. *Pitfall:* the redesign kept the shipped detector's logistic activation, harmless at its old depth; Glorot on a logistic stack raises ρ to the logistic's own ceiling, a quarter, and the attenuation stays hopeless. You cannot initialise your way out of the activation function: change both, ReLU with He, and pass `nonlinearity="relu"` to `kaiming_normal_` rather than its leaky default.

5. **Dead units.** A ReLU whose pre-activation is negative for every window has gradient exactly zero, forever, and the network trains around it with part of its capacity switched off. On a device every unit costs battery on every window of audio, so a dead one is a cost with nothing behind it: the forward probe counts them, per layer, before training and after, and the repairs are one idea, a non-zero slope on the negative side — leaky ReLU at no cost, ELU at an exponential per unit. *Pitfall:* the run reports a number and nobody asks how many units took part. A layer full of dead units after the first epochs usually means a step that threw the pre-activations negative, so read the count beside the learning rate; and the difference between ReLU and its variants was a few points on one seed today, so read it against the ablation's spread.

6. **Normalisation, and the bill on a device.** Initialisation gets ρ right once; normalisation makes it right at every step, standardising each layer's inputs and then learning γ and β.

Batch normalisation uses the batch's statistics in training and running averages in evaluation — two different functions — and on the device there is no batch, one window at a time; layer normalisation standardises within the row and is the same function in both places. *Pitfall:* added as a reflex to a network that no longer needed it, batch normalisation made it worse today, over five seeds, and it brings back the missing-`eval()` failure in a harder form, since in training mode it is deterministic for a fixed batch, so running the evaluation twice finds nothing and the device ships a function nobody validated. Add it only when the probe still shows ρ off one, and ask outright whether `eval()` was called.

7. **Clipping, the optimiser and the schedule — only now.** Clipping rescales the whole gradient when its norm exceeds a threshold, a defence against a ρ above one; look at the distribution of norms before choosing it. Compare optimisers only on the repaired network, each at a learning rate that suits it; a schedule needs to know how long the run is, so when the cluster's budget changes the epoch count the schedule changes silently with it. *Pitfall:* clipping applied to a vanishing gradient buys nothing, and a clip set below the median norm acts as a step-size limiter rather than a safety net — a gain for the wrong reason. If you want a smaller step, set a smaller learning rate and say so; the spread between optimisers today was far smaller than the effect of the initialisation.

8. **Prove it, then release it.** Each row of the ablation adds one change to the row above, on the fixed seed and subset; the columns are the test accuracy, its spread over a few seeds, and the change from the row above. A change smaller than the spread is written *noise*: today the one-cycle schedule's point sat inside its spread, batch normalisation cost points, and one row — ReLU with He — was worth more than the whole ladder. The winning row goes over the air to a small share of devices first, read against the shipped detector on misses and on false wakes per hour of audio, and the on-call engineer watches the false wakes, because each one is audio that left a home. *Pitfall:* the engineer changes the initialisation, adds batch normalisation and a schedule in one commit, the network trains, and the gain is credited to the fashionable change. "Should I add batch normalisation?" is malformed until you know which failure you have; the table is what makes it well formed, and a row that moves less than the spread is deleted before it reaches a device.

9. **Ship it.** Controls — loop, data, metric, the shipped detector → one seed, one subset → forward probe, down the batch → backward probe, ρ from the slope, logged per epoch → initialisation matched to the activation → dead units counted → normalisation only if ρ is still off one → clipping, optimiser, schedule, on the repaired network → the ablation, one change per row → a share of devices, read on false wakes.

8 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 10's deck and in the exercise book.

1. One layer multiplies the standard deviation of the backward signal by $\rho = \sqrt{n_{\text{out}} \operatorname{Var}(w)\, \mathbb{E}[\varphi'^2]}$. State what L layers do to it, and why only $\rho = 1$ survives depth. [5]

2. Preserving the forward signal asks for $\operatorname{Var}(w) = 1/n_{\text{in}}$ and preserving the gradient asks for $1/n_{\text{out}}$. State when the two agree, and what Glorot does when they do not. [4]

3. He initialisation doubles the weight variance for ReLU. Derive the factor of two in one line. [4]

4. A gradient profile that is a straight line on a log axis tells you something a curved one does not. State what, and why it identifies the cause. [4]

5. Gradient clipping is applied to a network whose gradients are fifteen orders of magnitude too small. Predict the effect, and say what the attempt reveals about the diagnosis. [4]

Summary

A twenty-layer stack sat at chance while nothing raised. Instrumenting it — which is what owning the loop bought — showed the gradient shrinking by a constant factor per layer, a straight line on a log axis, so one number characterises the whole network: $\rho = 0.1593$, compounding to 1.45×10^{15} over nineteen steps.

The first probe was wrong, and the notes keep it. `h.std()` spans the whole tensor and reads 0.07 in a network that has stopped computing; the spread down the batch falls by sixteen orders of magnitude. And a saturation threshold 6.3 standard deviations from the mean returned 0.000 at every depth, which was arithmetic rather than evidence.

One recursion explains it: $\mathrm{Var}(z) = n\,\mathrm{Var}(w)\,\mathrm{Var}(a)$, with the fan-in forwards and the fan-out backwards. The two are the same condition only on a square layer, Glorot takes their harmonic mean, and He doubles it because ReLU discards half the variance. The per-layer factor compounds over the depth, so only $\rho = 1$ survives it.

Six repairs, each measured, then stacked in a ladder over five seeds: the step worth +31.00 points is ReLU with He, against a typical spread of 0.59. Batch normalisation *cost* 4.33 points on a network that no longer needed it, and the 1-cycle schedule's +1.05 against ±1.20 does not clear its own spread.

The closing design exercise turned that into the repair of a voice-assistant company's wake-word detector in nine steps — controls first, a forward probe that can fail, the backward probe for ρ, initialisation matched to the activation, dead units counted, normalisation only when still needed, clipping and the optimiser only on the repaired network, one change per row of the ablation and one share of devices first — each step named for the pitfall it avoids.